Build Community-Taxon-Level Trait Table

Description

Builds a wide-format trait table where rows correspond to community taxon names (as they appear in pollen/community data) rather than species names. For each trait observation the function looks up the full taxonomic lineage of the source species and maps it to every community taxon that appears anywhere in that lineage (kingdom -> species). Trait values are then aggregated (median) per '(community_taxon, trait_domain)' and pivoted to a wide matrix ready for 'assign_functional_type_clusters()'.

This correctly handles pollen taxa resolved only to family or higher rank: "Asteraceae" in the community data will collect the median across all trait species whose lineage contains Asteraceae.

Usage

```
build_community_taxon_trait_table(
  data_trait_records,
  data_trait_taxonomy,
  data_community_taxonomy,
  aggregation_method = "median",
  verbose = TRUE
)
```

Arguments

`data_trait_records`	A data frame of trait observations in long format. Must contain columns 'taxon_name' (character; the original trait species name, matching 'sel_name' in 'data_trait_taxonomy'), 'trait_domain_name' (character; trait domain label), and 'trait_value' (numeric; observed value).
`data_trait_taxonomy`	A wide-format taxonomic classification table with one row per 'sel_name' and columns 'sel_name', 'kingdom', 'phylum', 'class', 'order', 'family', 'genus', 'species'. Typically produced by 'build_classification_table()' and combined with manual overrides via 'build_combined_classification_table()'. 'sel_name' must match the 'taxon_name' values in 'data_trait_records' (after trait QC and correction).
`data_community_taxonomy`	A wide-format taxonomic classification table for the community (pollen) taxa in this project / spatial unit. Must contain a 'sel_name' column (the pollen taxon name as it appears in the community data) and the seven rank columns 'kingdom', 'phylum', 'class', 'order', 'family', 'genus', 'species'. Typically the output of 'build_combined_classification_table()' applied to the pollen community.

	'sel_name' provides the allow-list of community taxa; each taxon is resolved to its finest available rank (via 'resolve_classification_to_finest_rank()') to look up matching entries in 'data_trait_taxonomy'. This correctly maps "Betulaceae Undiff" -> canonical "Betulaceae", "Betula Nana-Type" -> canonical "Betula", etc.
`aggregation_method`	A single character string specifying how trait values are aggregated per '(community_taxon, trait_domain_name)' pair. Must be one of '"median"' (default) or '"mean"'.
`verbose`	A single logical. If 'TRUE' (default), progress messages reporting coverage statistics are printed via 'cli'.

Details

Algorithm:

1. Apply 'resolve_classification_to_finest_rank()' to 'data_community_taxonomy' to obtain a canonical name for each pollen taxon — e.g. "Betulaceae Undiff" -> "Betulaceae", "Betula Nana-Type" -> "Betula". This handles all pollen suffixes ("Undiff", "-Type", "Subg", "Sect", "Cf") without any string manipulation.

2. Pivot the seven rank columns of 'data_trait_taxonomy' (traits-side) to long format. Retain only rows whose rank value matches a canonical name from step 1.

3. Join the community canonical-name table to the trait classification table on 'canonical_name' ('many-to-many'), producing a mapping '(pollen_taxon, canonical_name, trait_species)'.

4. Inner-join to 'data_trait_records' on 'taxon_name == trait_species', giving one row per '(pollen_taxon, trait_domain_name, observation)'.

5. Aggregate: median (or mean) of 'trait_value' per '(pollen_taxon, trait_domain_name)'.

6. Pivot wide: 'taxon_name' (pollen taxon) × trait domain columns.

Value

A tibble with one row per community taxon that could be matched to at least one trait observation. Contains a 'taxon_name' column (the original pollen taxon name from 'sel_name' column in 'data_community_taxonomy') plus one numeric column per distinct 'trait_domain_name'. Community taxa with no trait match are silently absent (not filled with 'NA' rows). The column order is 'taxon_name' followed by trait domain columns in alphabetical order.

See Also

[build_classification_table()], [build_combined_classification_table()], [resolve_classification_to_finest_rank()], [assign_functional_type_clusters()], [classify_to_functional_type()]